

Architecture

Decisions, trade offs and production considerations arising from the ledger core in this repository. Where a claim is about the code, the file is named.

1. Append only at scale

What breaks first is the balance query, and it breaks quadratically

`Ledger.balanceMinor` in `src/modules/ledger/ledger.ts` has no cache and no running total. It filters the account's entire history and sums it, and every balance in the system comes from that one function.

The cost is not the sum. It is who calls it, and how often. `assessOverdraftFees` walks every day in the window and calls `balanceMinor` once per day, at every day close:

```
days_to_reassess x entries_in_account_history
```

Both grow with time. The window is not six days in production. It is every day the account has been open, because a backdated entry can reach any of them. So the per close cost grows linearly in account age, and the entry count it scans grows linearly too. **Total cost is quadratic in account age.** Ten events over six days hides this completely; at 100 times the volume it is 10,000 times the work in a batch window that must finish before the business opens. Interest capitalization has the same shape, and so does a projection rebuild, which is the operation append only is supposed to make cheap.

Storage is not the problem. Storage is the cheapest part of an append only design and the one people worry about first.

Where the design accumulates unbounded state

The restatement window. Nothing limits how far back a `valueDate` may reach, so no day is ever finished. This is the root cause of the quadratic cost above and of most of section 2.

The hold register. `HoldRegister` has no expiry, so an authorization never settled and never voided holds funds forever. This replay hides it: the only unsettled authorization was declined, so no hold survives the window.

The event log and the ledger. Unbounded by design, and correctly so. These are history. The problem is not that they grow. It is that everything is recomputed from them.

Derived state recomputed rather than stored. Every closing balance is recalculated on demand. That is what makes a balance always correct and never stale, and it is the same property that makes it expensive.

The cheapest structural change

A periodic balance snapshot, plus a sealed period flag. One table and one job.

```
snapshot(account_id, value_date) -> closing_balance, as_of_sequence
```

A nightly job writes a closing balance per account for the day just sealed. `balanceMinor` reads the nearest snapshot and applies only later entries, so the scan is bounded by entries since the snapshot rather than by the whole history. The sealed flag forbids a value date inside a closed period; anything reaching that far posts to the current open period as a prior period adjustment.

It is the highest leverage change available because one mechanism bounds four problems at once:

Problem	How the snapshot bounds it
Balance query cost	Scan from the snapshot, not from inception
Fee reassessment loop	Only days in the open period can move, so only those are reassessed
Interest recomputation	Accruals in a sealed period are final and are read, not recalculated
Projection rebuild time	Rebuild from the last snapshot, not from the first event

It needs no change to the append only log. The log stays the source of truth and the snapshot stays a cache, so a snapshot can be discarded and rebuilt whenever it is doubted. That is what makes the change cheap: it cannot corrupt anything, so it can run alongside the existing path before anything depends on it. The second change, once the first is in, is a TTL on the hold register.

One property is worth naming. No rule in this model reads two accounts, so the log shards cleanly on `account_id` with no coordination, and `test/order-independence.e2e-spec.ts` demonstrates the property that makes that safe. It stops being true the moment double entry arrives, because a balanced transaction spans accounts. Worth knowing before choosing the sharding key, not after.

2. Value dated entries in a UAE licensed bank

The operational surface

A back valued entry does not change one number. It changes every number derived from the days it reaches, and most of those have already been sent somewhere.

Statements already issued become wrong. The bank must choose between reissuing, which invites the question of what else changed, and an adjustment line in the current period, which is harder to read but leaves the audit trail intact.

Fees already charged may no longer be due, or may now be due. The fee engine here reassesses closed days for exactly this reason. A customer can therefore be charged today for a day that closed a week ago, which needs an explanation the system cannot currently generate.

Interest already credited is recomputed. Silently, because interest is derived. Silent is fine inside a window nobody has reported. Across a reported period it is a restatement.

Downstream consumers have already consumed. Transaction monitoring, credit decisioning, limit management and the warehouse feed have all read the old numbers, and each has its own idea of what a correction means.

The regulatory surface

Named because a value date can move a transaction across each one. Each needs verification against current CBUAE issuances before go live; the point is the shape of the exposure, not a compliance opinion.

Consumer protection. The CBUAE Consumer Protection Regulation (Circular 8/2020) governs fee disclosure, error correction and complaints. A fee charged retroactively sits directly in that surface, and a fee a later reversal shows to have been the bank's own error sits further into it. This implementation cannot tell those apart, which is the subject of its one failing test.

AML and CFT. Federal Decree-Law No. 20 of 2018 and Cabinet Decision No. 10 of 2019 make monitoring date sensitive. Back valuing moves a transaction into or out of a window and can change whether a set of transactions constitutes structuring. Reporting clocks run from detection, so an alert raised today concerns activity dated weeks ago.

Regulatory reporting. Returns are period based. A value date landing in a submitted period changes a filed figure, which is a resubmission and a conversation rather than a routine correction.

Accounting. Under IFRS, interest recognition attaches to a period. An entry crossing a reporting boundary is a prior period adjustment under IAS 8 if material, and materiality is assessed in aggregate, so the exposure is the total volume of back valuing rather than any single entry.

VAT. Under Federal Decree-Law No. 8 of 2017, explicit fee based financial services are generally standard rated at five percent, unlike margin based products. A fee is a taxable supply with a tax point, and back dating or reversing one moves that tax point, potentially across a filed return.

Dormancy. The CBUAE Dormant Accounts Regulation keys off the date of last customer activity. A back valued entry changes that date retroactively, moving an account into or out of dormancy and, at the far end, unclaimed balance transfer.

Shari'ah compliance. This model is conventional. Interest on a credit balance and a flat overdraft fee do not transfer to an Islamic product, where the equivalents are profit distribution and a cost recovery charge on a different basis. A bank running both needs the accrual engine to be product aware.

The one control I would add before going live

A closed period lock, with back valuing beyond it requiring dual authorization and a reason code.

Value dates may not fall inside a sealed accounting period. Inside the open period, back valuing beyond T plus two business days requires maker checker approval, a reason from a closed list, and an immutable record of who approved it. Anything reaching further posts as a prior period adjustment in the current period, never as a silent rewrite of a closed one.

I would choose this over tamper evident hash chaining or a full bitemporal audit trail for three reasons.

It bounds how far a correction can reach, rather than recording where it reached. Every problem in this section follows from a correction reaching something already reported. A lock stops it reaching. An audit trail tells you afterwards, which is useful and is not the same thing.

It supplies the missing field. The reason code is the concept whose absence produces the one failing test. Once a reversal says why, the system can distinguish a bank error, where consumer protection rules require the fees back, from a legitimate customer return, where the account really was overdrawn. Today it applies one rule to both and is wrong about one of them every time.

It is the same mechanism as the scaling fix. The sealed period flag from section 1 and this lock are one feature. Getting the performance work and the regulatory control from one change is unusual and worth taking.

Runners up, in order: a TTL on the hold register, hash chaining the log, and a customer facing adjustment advice generated whenever a restatement crosses a statement boundary.

3. Authorization lifecycle

What this model supports

An authorization in `authorization.types.ts` has three states: approved, settled, declined. Reading the paths in `event-handlers.ts` rather than the state list, there are **three** ways one ends other than a settlement for the amount held, and a fourth where it never ends at all.

1. Declined at creation. The hold would take available balance below zero, so no hold is created and the state is terminal. *Scenario:* the ordinary one, a customer at a point of sale without the funds, or with funds already committed to an earlier hold. *Mandated, and implemented:* record the decline with the balances that produced it. It is an output an operator needs to see, not an error to swallow, and storing it lets a later settlement distinguish an authorization that was refused from one never requested.

2. Refused as a duplicate identifier. A second authorization under an identifier already in the register is refused and never becomes an authorization. *Scenario:* a retry. The acquirer did not receive the response, or a queue redelivered the message. *Mandated, and implemented:* refuse, and leave the existing authorization untouched. Treating the retry as new creates a second hold for the same purchase and locks twice the money. Note what this is not: refusal, not idempotency. A genuine idempotent path would return the original outcome so the retry succeeds.

3. Settled for an amount other than the amount held. The authorization terminates and the whole hold is released, whatever the difference. *Scenario:* routine, and what happens in this replay. A restaurant bill before a tip, a fuel pre authorization, a basket that changed before capture. *Mandated:* the implemented behaviour is only conditionally right. Releasing the entire hold is correct for a single presentment product and wrong for a multi presentment one, where a hotel folio or a split shipment presents again and the residual should stay held. Set the release policy per product, and treat a global choice as a defect whichever way it is set.

4. It never ends. An approved authorization never presented holds funds indefinitely. No expiry, no acquirer void, no sweep. *Scenario:* the merchant abandons the sale, the terminal drops after approval, or a pre authorization is never captured. *Mandated:* this path should not exist, so the mandate is the expiry below. The gap is invisible here because the only unsettled authorization was declined.

One guard keeps a terminated authorization terminated: a settlement against an already settled authorization is refused rather than posted, so a duplicate presentment cannot debit twice.

What production must mandate

Four endings this model has no path for, plus one case that is their mirror.

Expiry. The merchant never presents. Mandate a TTL per product, not one global value: card present retail commonly around seven days, hotel and vehicle rental thirty, fuel at an automated pump much shorter. On expiry append a release event and free the hold, never mutating the authorization in place. A settlement after expiry is then a force post and goes down that path rather than reviving a dead hold.

Acquirer initiated void or reversal. The merchant cancels at the terminal, or an estimated authorization is replaced by a final one. Mandate immediate release, full or partial, on receipt, and no further settlement against that identifier. This is the cheapest hold to release and the one customers notice most when it is not, because they are standing at the counter.

Over settlement beyond tolerance. The presentment exceeds the hold. Restaurant tips, fuel dispensed after the pre authorization and currency movement all do this legitimately. Mandate a tolerance per merchant category, commonly fifteen to twenty percent for restaurants and fuel and zero for most others. Within tolerance, post and release. Outside it, post anyway, because the issuer is generally obliged to honour the presentment, and raise an exception so the excess is investigated rather than absorbed.

Settlement with no authorization at all. Not an ending, because there is nothing to end. Included because it mirrors every ending above, and because this model has a path for it and takes the wrong one. Offline and floor limit transactions, chip fallback, stand in processing during an issuer outage, late presentment after expiry. This implementation refuses these, because the brief requires it, and no real issuer could. Mandate posting to a suspense account with an exception raised, then pursue chargeback rights if the authorization is genuinely absent. Never silently decline: the acquirer is unpaid and the customer has the goods.

Account state change between authorization and settlement. A freeze, a sanctions match, a court order, a closure, or the death of the customer. Mandate that the existing hold survives, because those funds are already committed, while new authorizations are declined. A settlement against a frozen account still posts, with the freeze enforced at the balance level rather than by dropping the posting, so the bank does not create an unreconciled difference with the scheme. Escalate to compliance rather than resolving in the ledger.

One more sits outside the lifecycle but ends the economics: a **chargeback** after settlement, which reverses the posting under scheme timelines entirely separate from anything in this model.

4. What was cut, and what each cut defers

Ordered by the size of the risk, not the size of the change.

Double entry. Single entry per customer account, with no contra accounts for fee income, interest expense, settlement clearing or suspense. *Defers:* no trial balance, no way to prove value is conserved across the book rather than within one account, and finance cannot see fee income or interest expense at all. A ledger that cannot be balanced is a record, not a ledger. This is the largest cut here.

Closed periods and a back value limit. *Defers:* everything in section 2. Every statement and every regulatory return stays permanently provisional.

Snapshots. *Defers:* the quadratic day close from section 1, and a projection rebuild that goes from seconds to hours.

Idempotency. No event carries a key and nothing deduplicates. *Defers:* a retried delivery double posts. In a system fed by a queue with at least once delivery, this is not an edge case. It is routine.

Concurrency control. Single threaded, no locking, no optimistic versioning. *Defers:* two authorizations arriving together both read the same available balance and both approve, and the account goes overdrawn past the limit meant to prevent exactly that.

Authorization expiry. *Defers:* holds leak, and customer funds are locked with no event that will ever free them.

Reversal reason codes. *Defers:* the consumer protection exposure in section 2, and the one failing test.

Tamper evidence. The log is append only by construction, with frozen records, but it is not hash chained or signed. *Defers:* an append only log with no tamper evidence is not an audit record, whatever the code does. Cheaper to add before there is history to migrate.

A business calendar. No weekends, no UAE public holidays, no timezone, no cutoff. *Defers:* accruals and value dates diverge from the business calendar, and the divergence compounds. The UAE weekend moved from Friday and Saturday to Saturday and Sunday for the federal sector in January 2022, so the rule is not even a constant.

A day count convention. A flat rate per day, with no Actual/365, Actual/360 or 30/360. *Defers:* the accrual will not agree with product terms or with treasury, and the difference is small per day and systematic.

Debit interest. An overdraft is priced by a flat fee only. *Defers:* an overdraft is mispriced, and regressively, since a small overdraft pays the same fee as a large one.

A multi currency fee schedule. Only AED is priced. *Defers:* an account in another currency cannot be charged, which the code raises on rather than guessing. Honest, and still a gap.

Access control and segregation of duties. No authentication, no authorization, no maker checker. *Defers:* every control in section 2 assumes somebody can be prevented from doing something, and nothing here can prevent anything.

Observability. No metrics, no structured logs, no reconciliation report. *Defers:* the first production incident is diagnosed by reading source code.

Event schema versioning. *Defers:* the first change to an event shape makes the historical log unreadable by the current code, which is the failure that turns an append only log from an asset into a liability.

Persistence, and everything downstream of it. No storage, no transactions, no recovery. Out of scope by instruction, and named because every cut above interacts with it.